Assignment 1 - Git & GitHub Fundamentals

Report

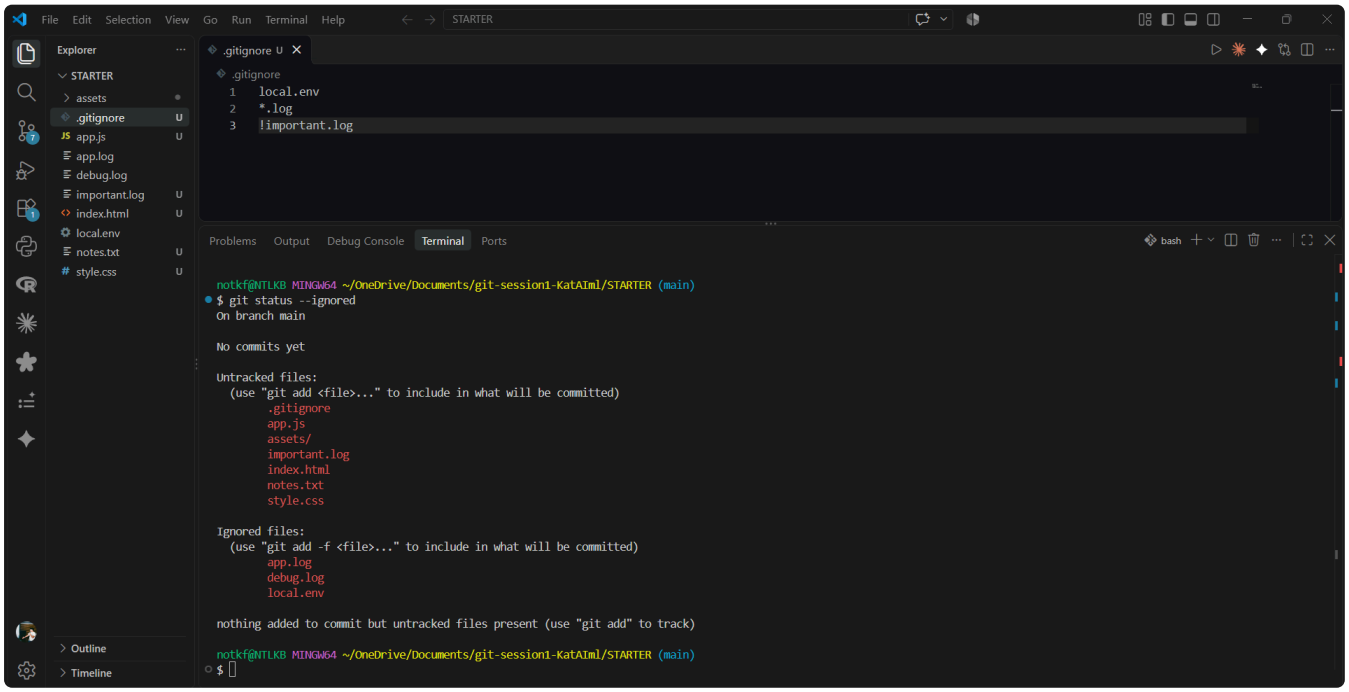
- ***Full Name:*** Khalil Berjaoui
- ***Student ID:*** 124433
- ***Course Code & Section:*** CSC 3324 02
- ***GitHub Username:*** KatAIml

1. Repository Setup

I copied the STARTER files into a folder called `git-session1-KatAIml` and ran `git init` to make it a Git repo (branch `main`). Then I made a `.gitignore` file with three lines:

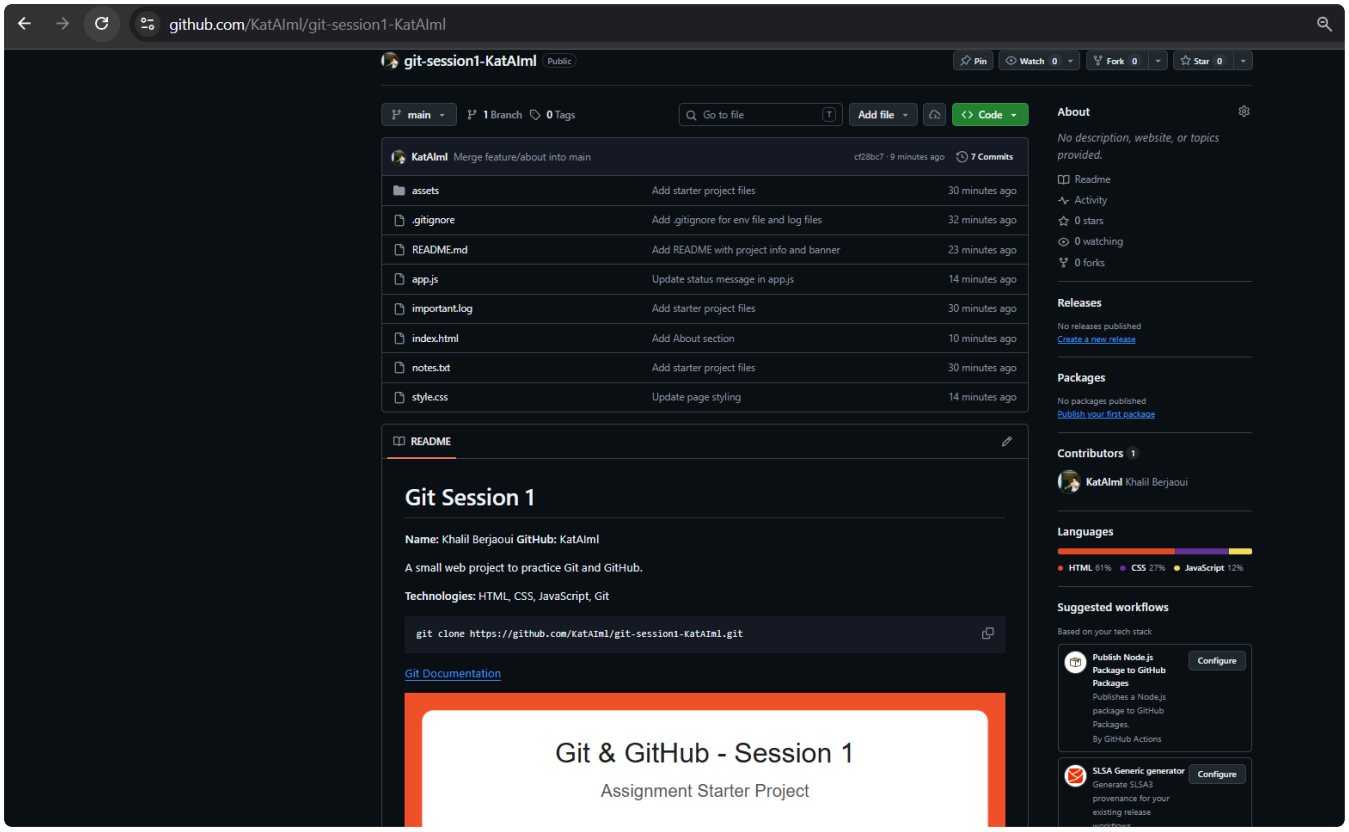
- `local.env`: keeps my local settings and keys out of Git
- `*.log`: ignores log files like `app.log` and `debug.log`
- `!important.log`: an exception, so `important.log` is still tracked

`git status --ignored` shows it works: the two log files and `local.env` are ignored, but `important.log` is not.



2. README & Commit History

My README has my name, GitHub username, a short description, the technologies, a `git clone` code block, a link to the Git docs and the banner image.



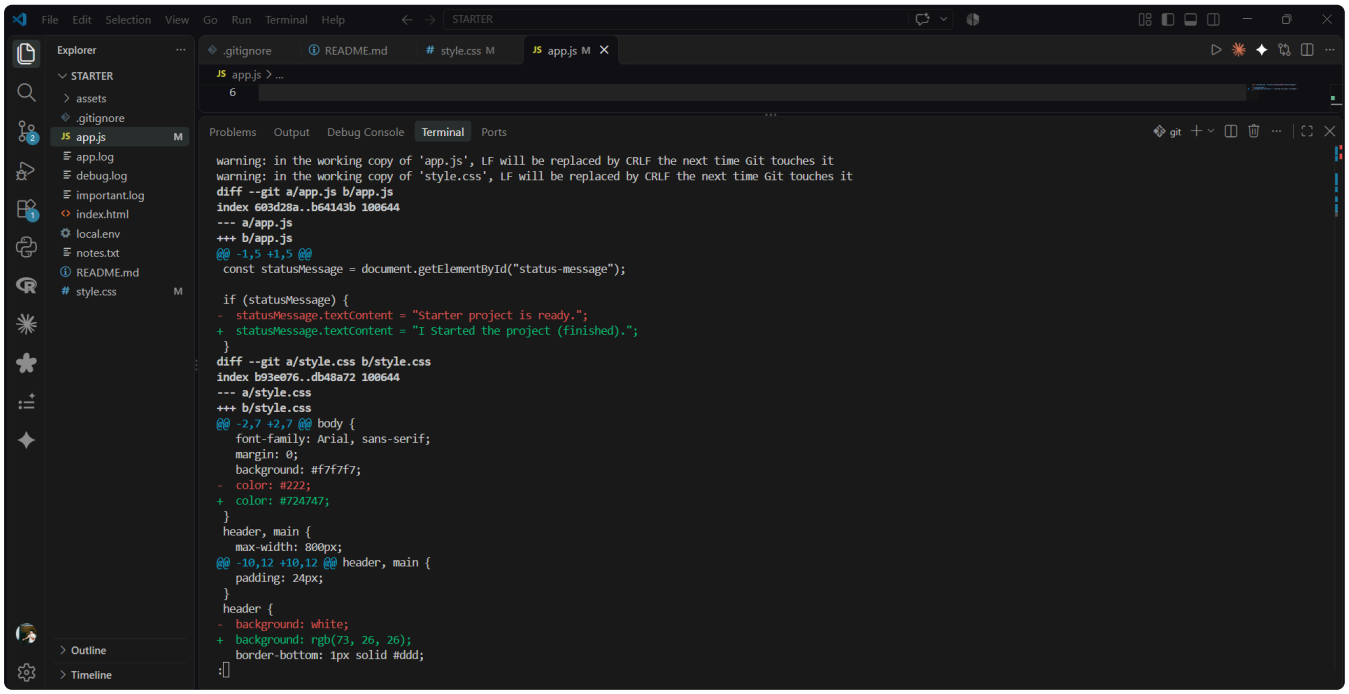
I kept each commit to one change, with a message that says what it does:

Commit	Message	What changed
263f0b3	Add .gitignore for env file and log files	`.gitignore`
fb46d3b	Add starter project files	HTML, CSS, JS, assets, `important.log`, `notes.txt`
896d060	Add README with project info and banner	`README.md`
84bb160	Update page styling	`style.css`
ce23601	Update status message in app.js	`app.js`
ba4ddae	Add About section	`index.html` (on `feature/about`)
cf28bc7	Merge feature/about into main	merge commit
f4969dc	Update README.md	`README.md` (edited on GitHub)

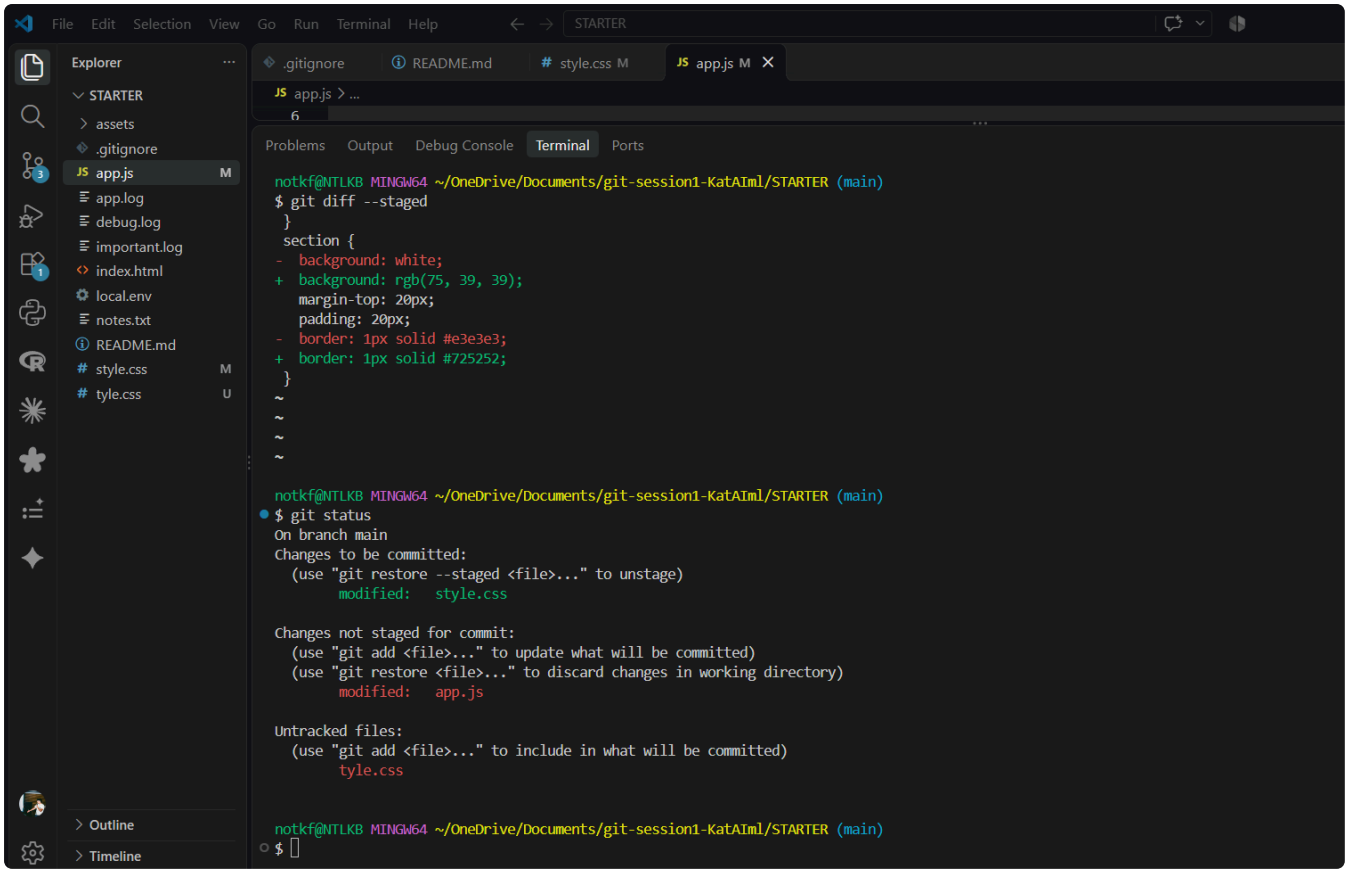
So that's 6 normal commits and 1 merge done locally, plus 1 README edit made on GitHub.

3. Selective Staging

I changed two files at the same time: the colors in `style.css` and the message in `app.js`. `git diff` showed both as unstaged changes.



Then I staged only one of them with `git add style.css`. `git diff --staged` showed just the CSS change, and `git status` showed `style.css` ready to commit while `app.js` was still not staged. I committed the CSS, left `app.js` modified, and committed it later in its own commit.



4. Branch & Merge

I made a new branch with `git switch -c feature/about`, added an About section to `index.html` and committed it. Then I went back with `git switch main` and ran `git merge --no-ff feature/about`.

`--no-ff` makes Git create a merge commit, so the branch still shows in the history graph instead of disappearing.

```
notkf@NTLKB MINGW64 ~/OneDrive/Documents/git-session1-KatAIml/STARTER (main)
$ git log --oneline --graph --all
*   cf28bc7 (HEAD -> main) Merge feature/about into main
|\
| * ba4ddae (feature/about) Add About section
|/
* ce23601 Update status message in app.js
* 84bb160 Update page styling
* 896d060 Add README with project info and banner
* fb46d3b Add starter project files
* 263f0b3 Add .gitignore for env file and log files
```

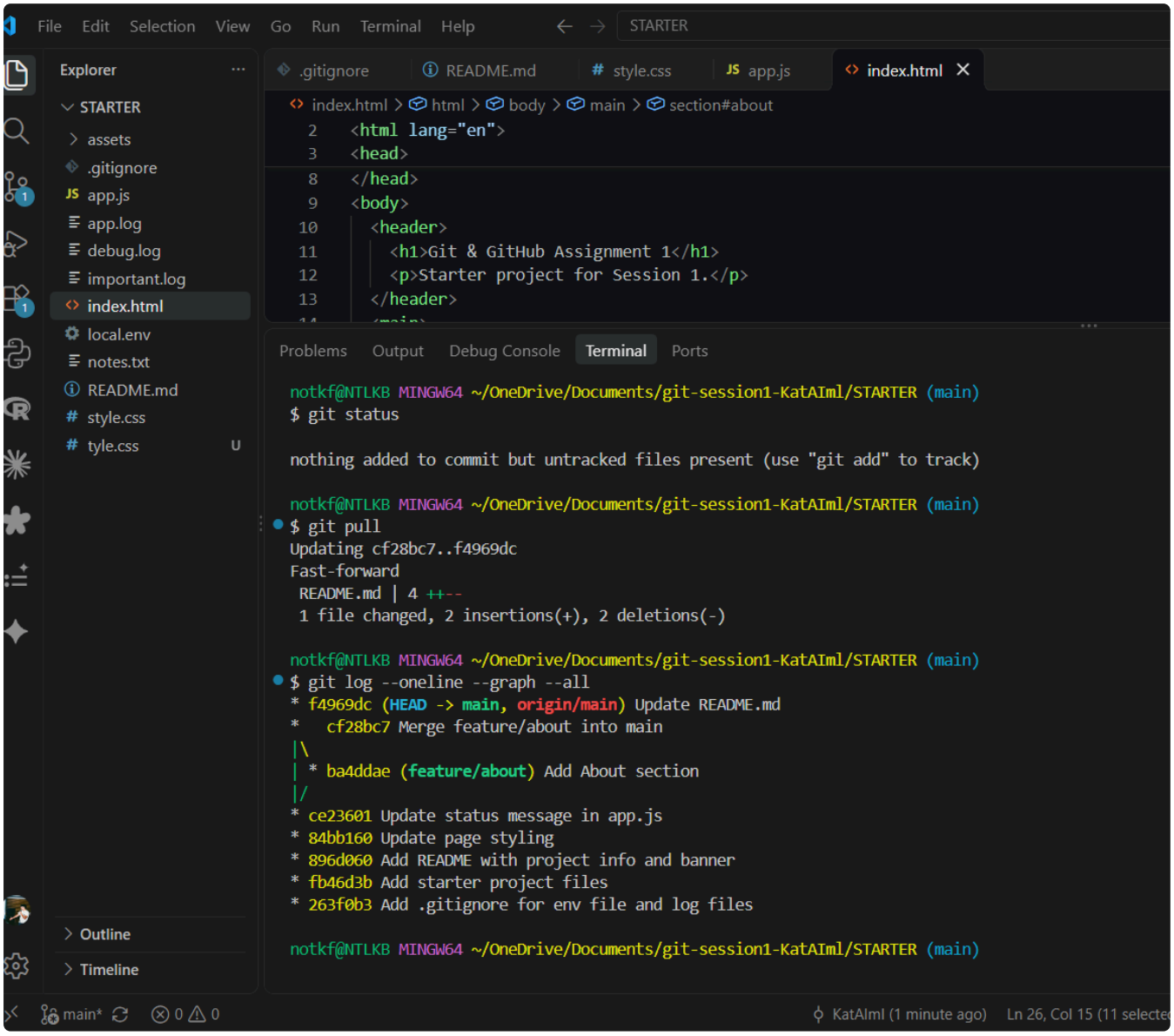
5. Remote Changes

I made an empty public repo on GitHub, linked it with `git remote add origin <url>`, and uploaded `main` with `git push -u origin main`.

- **Commit vs push:** commit saves my changes on my computer. Push sends them to GitHub.
- **Fetch vs pull:** fetch downloads the new commits from GitHub but doesn't touch my files. Pull downloads them and updates my files too.

To try it, I edited the README on GitHub. After `git fetch`, `git status` said I was 1 commit behind `origin/main`, but my README file hadn't changed yet.

After `git pull`, my local `main` caught up, and the log shows `main` and `origin/main` on the same commit.



6. Fair Use of AI

none as i dont need it for this job (part time job knowledge)

7. Sources & Reflection

Sources

- Session 1 slides and practice files
- Git documentation
- gitignore documentation

Issues I solved

- I typed the `.gitignore` lines into the terminal by mistake and got `command not found`. They needed to go inside the file.
- `git diff` opened a screen ending with `:`. Pressing `q` closes it.
- Git showed "LF will be replaced by CRLF" warnings. That's just Windows line endings, not an error.

Reflection

Nothing in this assignment was really hard for me, because I already use Git and GitHub every day in my part-time job at Neocedrus. If Git does something strange, I first run `git status` to see my branch and what's staged, then `git log --oneline --graph --all` to compare with `origin/main`.